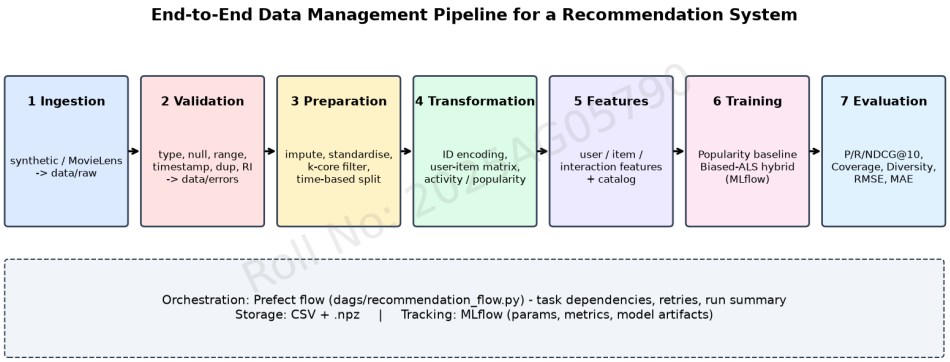
End-to-End Data Management Pipeline for a Recommendation System

DM4ML Assignment 2 | Roll Number: **2025AG05790**

1. Problem and data

The pipeline recommends items to users from past ratings and item metadata. Because the course allows synthetic data, a seeded generator (*src/ingestion*) creates 1000 users, 600 items and ratings (1-5) with timestamps from a latent-factor model plus Zipf item popularity, so personalisation signal and popularity bias both exist. It also injects ~2% dirty records of each type so validation has real defects to catch. A loader for MovieLens-100K (*data.source: movielens*) is included; the same pipeline runs unchanged on it.

2. Pipeline design and tools



Validation (pandas rules): missing values, data types, rating range 1-5, timestamp parsing and range, exact duplicates, duplicate keys and referential integrity (interaction -> user/item). Rejected rows go to *data/errors/*.csv* with a *reason* column; counts are in *reports/validation_report.md*. Fixable issues (missing age, negative price) are kept and imputed in preparation rather than discarded.

Preparation / transformation: text standardisation, median imputation, iterative k-core filter (>= 5 interactions per user and item), and a **global time-based split** (newest 20% of interactions form the test set) so no future data leaks into training. Derived outputs: ID maps, sparse user-item matrix (.npz), user activity summary and item popularity table.

Feature management: a custom file-based feature layer. User, item and interaction features are computed from the training split only and read back through *get_user_features / get_item_features*, so training and prediction use identical definitions (catalog: *reports/feature_catalog.md*). Chosen over Feast to keep the project light and fully reproducible offline.

Training / tracking: (1) popularity baseline; (2) biased matrix factorisation fitted by ALS, scored with a hybrid of predicted rating and log-popularity. The popularity weight is chosen on a validation slice (latest 15% of the *training* data), never on test. MLflow logs parameters, metrics and the pickled models.

Orchestration: a Prefect flow chains the seven tasks with explicit dependencies and retries on ingestion, validation and training; seeds are fixed, so reruns reproduce the same results. A plain-Python runner (*run_pipeline.py*) is the fallback.

3. Data quality results

Table	Rows in	Clean	Rejected
users	1005	1000	5
items	600	600	0
interactions	37664	33644	4020

Violations found: users: duplicate user_id = 5; users: missing age (kept, imputed later) = 10; items: negative/non-numeric price (kept, imputed later) = 8; interactions: missing user/item/rating = 754; interactions: rating outside range = 757; interactions: unparseable timestamp = 495; interactions: timestamp out of range = 256; interactions: exact duplicate rows = 688; interactions: user_id not in users (RI) = 339; interactions: item_id not in items (RI) = 731. Note that rejected interaction counts exceed the injected-defect count slightly because a row is rejected once, under the first rule it breaks.

4. Model results

model	Precision@10	Recall@10	NDCG@10	Coverage	Diversity	RMSE	MAE
popularity_baseline	0.1502	0.3895	0.2720	0.0858	0.8144	1.2172	1.0186
biased_als	0.1797	0.4743	0.3726	0.1381	0.8234	0.8534	0.6813

Evaluated on the time-based test set for 806 users with at least one relevant (rating >= 4) future item; already-rated items are excluded from recommendations. The hybrid ALS model improves NDCG@10 from 0.272 to 0.373, Recall@10 from 0.390 to 0.474, coverage from 0.086 to 0.138, and cuts RMSE from 1.217 to 0.853. An early run with pure predicted-rating ranking (no popularity term) had much better RMSE but *worse* ranking than the baseline, because future interactions are dominated by popular items; this is why the hybrid score was introduced and tuned on validation data.

5. Orchestrated run

All tasks completed: ingest_data (0.26s), validate_data (0.35s), prepare_data (0.27s), transform_data (0.27s), build_features (0.31s), train_models (8.15s), evaluate_models (0.52s).

6. Limitations

(a) Results are on synthetic data, whose structure favours the model family used; absolute numbers will differ on real data such as MovieLens. (b) One global time split and a single seed: no confidence intervals. (c) ALS is a from-scratch NumPy implementation (slower than libraries such as Implicit) and treats ratings as explicit feedback. (d) The feature layer is file-based, with no online low-latency store or point-in-time joins beyond the train-only rule. (e) Validation is rule-based; no statistical drift checks. (f) Diversity uses only category vectors. (g) Cold-start users and items are not handled: test rows without training history are dropped.

Roll No. 2025AG05790